CPS721: Assignment 3

Due: October 29, 2026, 11:59pm
Total Marks: 100 (worth 4% of course mark)
In groups of no more than 3

Late Policy: The penalty for submitting even one minute late is 10% for each day late. Assignments are not accepted more than 72 hours late.

Clarifications and Questions: Please use the discussion forum on the D2L site to ask questions. These will be monitored regularly. A Frequently Asked Questions (FAQ) page will also be created. Only ask questions by email if it is specific to your submission. Any questions asked with less than 24 hours left to the deadline will not be answered.

PROLOG Instructions. Your code must run on ECLiPSe 7.1. If you cannot install this on your own machine, test it on the lab machines or the department moon servers.

You are NOT allowed to use ";" (disjunction), "!" (cut), "->" (if-then), `setof`, `bagof`, `findall`, or any other language features and library predicates that are not covered in class. Similarly, you should only use "=" and "is" for equality, and "not" for negation. Do NOT use "=:=", "\=", or other such commands. You will be penalized if you use these commands/features. *Note that you lose all your marks if you solve this homework using techniques or predicates not covered in class.* You are only allowed to use ";" to get additional responses when interacting with PROLOG from the command line (this is equivalent to using the "More" button in the ECLiPSe GUI).

Note, this also means you are NOT allowed to use the built-in `member` or `append` predicates. However, you can reimplement these. If you do so, make sure to give it a new name that doesn't clash with the built-in predicates.

Academic Integrity and Collaboration Policy: Any submission you make must be the work of your group and your group alone. You are also not allowed to post course materials (including assignments, slides, etc.) anywhere on the web, and you are also prohibited from using generative AI systems or online "help" websites for completing assignments.

SUBMISSION INSTRUCTIONS: Submit ONE zip file called `assignment3.zip` containing 6 files:

```
assignment3_submission_info.txt
q1a_generate_and_test.pl      q1b_interleaving.pl         q1_puzzle_session.txt
q2_exams.pl             q2_exams_session.txt
```

Optionally, if you would like to solve Part 3 with a bonus question, then your zip file has to include also two other files: `q3_chess.pl` and `q3_chess_session.txt` All these files have been given to you and you should fill them out using the format described. Ensure your names appear in all files — especially `assignment3_submission_info.txt` — and your answers appear in the file sections as requested. Do NOT edit, delete, or add any lines that begin with "`%%%%% SECTION:`". These are used by the auto-grader to locate the relevant code. You will lose marks if you edit such lines.

Your submission should not include any other files than those above. You will be penalized for using compression formats other than `.zip`. Submissions by email will not be accepted.

You may submit your solution multiple times. Doing so will simply overwrite your previous submission. We will therefore always mark the last submission your group makes.

Make sure the files are saved in plain text and readable on Linux machines.

1 Crypt-Arithmetic [50 marks]

Use Prolog to solve the following crypt-arithmetic puzzle involving **multiplication**:

```
          H O W
      *   C A W
-----------------
          W H Y
   +    A M Y .
       C O T . .
-----------------
       M O N E Y
```

This notation means that HOW * CAW = MONEY, and the words between the bars correspond to performing the usual multiplication algorithm. That is HOW * W = WHY, HOW * A = AMY, HOW * C = COT and WHY + 10*AMY + 100*COT = MONEY. For example, the calculation 123 * 12 would be shown as follows when using the same notation:

```
          1 2 3
      *     1 2
-----------------
  +       2 4 6
        1 2 3 .
-----------------
        1 4 7 6
```

Each letter stands for a distinct digit and leading digits are not zeroes. You cannot make any other assumptions, and you cannot encode any of your guesses in your program. Follow the techniques from class literally. You will solve this problem in two parts.

a. [20 marks]

First, try to solve this problem using the *pure generate and test* technique, without any interleaving, and notice how much time it takes. You have to write your solution in the given file `q1a_generate_and_test.pl` Please follow the format indicated in that file, including having comments briefly explaining the meaning of your variables and having comments explaining your program. Your program should include the rule for predicate `solve(L)`, which takes in your list of variables `L` and finds an assignment that solves the puzzle. You should also write the rule for *print_solution(L)* similar to what was explained in class and make sure the TA can easily read the output. You have flexibility on the exact format, provided that it is clear to the grader what are the values of all variables. Note that we have given you a predicate called `solve_and_print` which calls `solve(L)`, outputs the runtime, and also calls `print_solution(L)`. This is the main way we will interact with your program. Ensure that both the *solve(L)* and *solve_and_print* can be run.

Note that rule implementing `solve_and_print` includes the following code that uses the system construct *cputime* and determines how much computer time your computation takes

```
Start is cputime, solve(L), End is cputime, Time is End - Start
```

The value of `Time` will be the time taken. Copy the output from `solve_and_print`, including the time and solution, to the file `q1_puzzle_session.txt` in the correct section.

b. [30 marks] Next, solve this problem using a smart interleaving of generate and test. Again implement *solve(List)* and *print_solution(List)*, and add them to the file `q1b_interleaving.pl`. In this file, add comments which explain briefly the order of constraints you have chosen and why this has an effect on computation time. You can draw a dependency graph by hand and include a PDF image if you decide to use one (but it is not a requirement).

Find also how much time your program takes to compute an answer in this case. Again, include the results of your session in `q1_puzzle_session.txt`

2 The Exams Week [50 marks]

Five university classes - biology, calculus, history, literature, and physics - are holding their exams next week. The exams will take place over five consecutive weekdays Monday through Friday.

Each class has exactly one exam, and only one exam is scheduled per day. Furthermore, each exam takes place in a different classroom (Rooms 101, 102, 103, 104, and 105), and each class has a different class size (20, 25, 30, 35, and 40 students). The following partial information is provided about the day, classroom, and student count.

1. The history exam will take place exactly two days after the biology exam (for example, if biology is on Tuesday, history is on Thursday). The calculus exam takes place the day immediately following the history exam.
2. The classroom Room 105 hosts an exam exactly four days after Room 101. The exam in Room 102 is scheduled for the day immediately before the exam in Room 103, which takes place earlier in the week than the exam in Room 104.
3. The physics exam has 35 students. It takes place earlier in the week than the calculus exam, but it is not the first exam of the week.
4. The literature exam is scheduled for Room 101, but it does not have 20 students.
5. The largest class (40 students) has its exam in Room 105. The smallest class (20 students) has its exam the day after physics.

a. [**40 marks**] Your task is to write a PROLOG program to solve this problem using the smart interleaving of generate-and-test technique explained in class: your program has to compute the match between the courses and their exams, classrooms and student counts. *Do* **not** *attempt to solve any part of this puzzle yourself*, i.e., do not make any conclusions from the statements given to you. *You lose marks if your program implements your own reasoning.* To get full marks, you have to follow a design technique from class.
Hint: You might wish to introduce a predicate for weekdays. You can choose the set of variables in your program, so it is easier to formulate the constraints.

You are allowed to split any single clue into different constraints, and then reorder the hints and constraints - including any all_diff calls - as you see fit.

Explain briefly the ordering you have chosen (write comments in your program). Remember to implement all implicitly stated and hidden constraints. They must be formulated and included in the program to find a correct solution satisfying all constraints. You can use predicates from class in your program, but rename them. If you like, you can introduce any additional helping predicates.

Determine how much computer time your computation takes using `cputime` construct. Never try to guess part of solution by yourself: all reasoning should be done by your program. You have to demonstrate whether you learned well a program design technique. You will lose marks, if the TA finds that you embedded some of your own reasoning into your program.

You should implement both a *solve(List)* and *print_solution(List)* as described in question 1. These should be put in the file `q2_exams.pl`. Explain briefly the ordering you have chosen (write comments in your program file). Make sure that *solve_and_print* prints the answers returned by your program in an easy to read format: you will lose marks if output is obscure.

b. [**10 marks**] You should also include your session testing your program in `q2_exams_session.txt`. This test should include information about the runtime. We should be able to query your program using both *solve(List)* and *solve_and_print* (no arguments).

Thus, your submission should include both `q2_exams.pl` and `q2_exams_session.txt`

3 Chess Tournament: BONUS [25 marks]

To make up for a grade on another assignment or test that was not what you had hoped for, or simply because you find this area of Artificial Intelligence interesting, you may choose to do extra work on this assignment. *Do not attempt any bonus work until the regular part of your assignment is complete. This part of the assignment is more challenging than questions in the regular part of your assignment.* If your assignment is submitted from a group, write whether this bonus question was implemented by all people in your team (in this case and by default bonus marks will be divided evenly between all students) or whether it was implemented by one student only (in this case only this student will get all bonus marks). Moreover, bonus marks will be given only if your group submits all solutions before the initial deadline on Thursday, October 29. Any correct solutions to bonus questions submitted late will not earn any bonus marks.

Five chess grandmasters - Alex, Beatrice, Charlie, Diana, and Ethan - recently played in a round-robin tournament. In other words, every player played exactly one game against every other player. The outcome for each game is either a draw (worth 1 point for both players), or a win for one player (worth 2 points for the winning player and 0 points for the losing player)

Your task is to write a Prolog program that finds the outcome of each game for each player, and the total scores based on the clues provided below. You have to write a Prolog program that implements precisely not only the given constraints, but also constraints that remain implicit. Never try to guess part of solution by yourself: all reasoning should be done by your program. You have to demonstrate whether you learned well a program design technique. Your implementation should follow exactly one of the design patterns explained in class. You will lose marks, if the TA sees that you embedded your own reasoning into your program. Points per individual game must satisfy all of the following constraints.

1. After all the games were finished, every player ended up with a *completely different unique total score*.
2. The champion (1st place) was ruthless and did not have a single draw in the entire tournament.
3. The runner-up (2nd place) put up an ironclad defense and did not lose a single game, except of the game with the champion.
4. The player in 4th place struggled to find a breakthrough and did not win a single game.
5. Alex finished ahead of Beatrice, who finished ahead of Charlie.
6. Diana won her match against Ethan.

a. [**20 marks**] Write a Prolog program `q3_chess.pl` that solves this problem using interleaving of generate and test technique considered in class.

You will need to be careful in your program regarding the order of constraints. Also, provide comments where in your program which constraint is implemented. Explain briefly the ordering you have chosen (write comments in you program file). You should implement both a *solve(L)* and *print_solution(L)* as described in question 1. Make sure that *solve_and_print* has no arguments and prints the answers returned by your program in an easy to read format: you will lose marks if output is obscure. Arrange your output similar to the predicate *print_solution(List)* discussed in class. The TA should be able to call both your predicate *solve(List)* and your predicate *solve_and_print* to see the solution.

It is up to you how to design your program as long as it follows the technique explained in class. The programs using the ECLiPSe libraries will not earn any credit. The following is one possible direction, but you can write your program differently.

You might wish to generate individual game scores for the players Alex, Beatrice, Charlie, compute their totals, compare them to satisfy the constraint in the clue (5), then generate the remaining few scores for Diana and Ethan, make sure they satisfy the constraint in (6), compute the remainign totals, and then check the constraint in (1). After that, your program can combine each list of individual game scores with the name of the player and the total into term. Finally, sort these terms by total, since we know all of them are different, thereby determine who are the winner, the runner-up and the 4th player, and finally check the remaining constraints (2), (3), (4). It is OK if the first solution printed by your program is correct. There is no need to find all correct solutions.

b. [**5 marks**] We should be able to query your program using both *solve(List)* and *solve_and_print* (no arguments). Make sure it is obvious from your output what the solution is. Find also how much time your program takes to compute an answer using the process described in the previous Part 1 of this assignment. Your Prolog session should be included in the file `q3_chess_session.txt` Thus, your submission should include both `q3_chess.pl` and `q3_chess_session.txt`